Bates Model Framework for Prediction Market Mispricing

Exploiting BTC Binary Option Contracts on Gemini

Project Specification & Development Outline

v1.0 — March 2026

1. Project Overview

Context, objectives, and scope of the system.

1.1 Problem Statement

Gemini offers binary prediction contracts of the form "Will BTC break price K by time T?" These contracts pay $1 if the condition is met and $0 otherwise, making them equivalent to digital (binary/cash-or-nothing) options on BTC/USD. The market prices for these contracts may deviate from their theoretical fair value as implied by the current BTC spot price, volatility surface, and jump dynamics. This project aims to identify and exploit such mispricings systematically.

1.2 Core Thesis

The Bates (1996) stochastic volatility with jumps (SVJ) model provides a richer description of BTC price dynamics than Black-Scholes or even Heston alone. By calibrating a Bates model to observable market data (spot price, implied volatility surface, historical returns), we can derive theoretically fair prices for binary contracts and trade when the Gemini market price diverges beyond a statistically significant threshold.

1.3 Objectives

1. Implement the Bates SVJ model calibrated to BTC/USD market data.
2. Derive fair binary option prices from the calibrated model via Fourier inversion or Monte Carlo.
3. Construct a real-time pipeline: spot data ingestion, model recalibration, contract pricing, and signal generation.
4. Define a trading strategy with explicit entry/exit rules, position sizing, and risk controls.
5. Backtest the strategy against historical Gemini contract data (or synthetic proxies).
6. Deploy for paper trading, then live execution via Gemini API.

2. Theoretical Foundation

Mathematical specification of the Bates model and its application to binary option pricing.

2.1 The Bates (1996) SVJ Model

The Bates model extends the Heston stochastic volatility model by adding Merton-style log-normal jumps to the asset price process. Under the risk-neutral measure Q, the dynamics are:

$$dS(t) = (r - q - \lambda\bar{m})S(t)dt + \sqrt{v(t)}\, S(t)dW_1(t) + S(t)dJ(t)$$

$$dv(t) = \kappa(\theta - v(t))dt + \sigma_h \sqrt{v(t)}\, dW_2(t)$$

$$dW_1 \cdot dW_2 = \rho\, dt$$

$$J(t)\text{: Compound Poisson, intensity } \lambda\text{, log-jump} \sim N(\mu_j, \sigma_j^2)$$

$$\bar{m} = \exp(\mu_j + \sigma_j^2/2) - 1 \quad \text{(compensator)}$$

2.2 Parameter Definitions

Parameter	Symbol	Description	Typical BTC Range
Spot price	S(t)	Current BTC/USD price	Market observed
Variance	v(t)	Instantaneous variance	0.3–1.5 (annualized)
Mean-reversion speed	κ	Rate v(t) reverts to θ	1.0–5.0
Long-run variance	θ	Equilibrium variance level	0.4–1.0
Vol of vol	σ_h	Volatility of the variance process	0.5–2.0
Correlation	ρ	Spot–variance correlation (leverage)	-0.8 to -0.2
Jump intensity	λ	Expected jumps per year	1–20
Jump mean	μ_j	Mean of log-jump size	-0.05 to 0.02
Jump vol	σ_j	Std dev of log-jump size	0.05–0.15

2.3 Characteristic Function

The Bates model admits a semi-analytic characteristic function φ(u) for log(S(T)), which has the Heston form augmented by a jump component:

$$\varphi(u) = \varphi_Heston(u) \times \exp(\lambda\, T\, [(1 + \bar{m})^{iu} \exp(-\mu_j u + \sigma_j^2 u^2/2) - 1 - iu\, \bar{m}])$$

This characteristic function enables pricing via Fourier inversion (Gil-Pelaez, Carr-Madan, COS method) without Monte Carlo simulation, making real-time recalibration feasible.

2.4 Binary Option Pricing

A cash-or-nothing binary call paying $1 if S(T) > K is priced as:

$$C_bin(K,T) = e^{-rT} \times Q(S(T) > K)$$

where Q(S(T) > K) is the risk-neutral probability recovered from the characteristic function via:

$$Q(S(T) > K) = 1/2 + (1/\pi) \int_0^\infty Re[e^{-iu \ln(K)} \varphi(u) / (iu)]\, du$$

For binary puts (paying $1 if S(T) < K): $P_{bin} = e^{-rT} - C_{bin}$.

3. Data Requirements

All data sources, schemas, and ingestion pipelines.

3.1 Data Sources

Data	Source	Frequency	Purpose
BTC/USD spot price	Gemini REST/WebSocket API	Real-time (tick/1s)	Model input S(t)
BTC options chain	Deribit API (primary BTC options venue)	5-min snapshots	IV surface calibration
Gemini prediction contracts	Gemini API / scraping	Real-time	Market price of target contracts
BTC historical OHLCV	Gemini / CoinGecko	1-min, 1-hr, daily	Historical calibration, backtesting
Risk-free rate	US Treasury / FRED API	Daily	Discounting (r)
Funding rates / basis	Deribit / Binance	8-hourly	Cost-of-carry proxy (q)

3.2 Data Pipeline Architecture

7. Ingestion layer: WebSocket connections to Gemini (spot, contracts) and Deribit (options). Fallback to REST polling.
8. Normalization: Unify timestamps (UTC), handle missing data, deduplicate.
9. Storage: Time-series database (TimescaleDB or InfluxDB) for tick data; PostgreSQL for contract metadata and trade records.
10. Feature store: Precomputed rolling realized volatility, jump statistics, and IV surface snapshots.

4. Model Calibration

Procedures to estimate and continuously update all Bates model parameters.

4.1 Calibration Strategy

A two-stage calibration approach balances robustness with speed:

Stage 1 — Historical estimation (offline, daily): Estimate jump parameters (λ, μ_j, σ_j) from historical BTC returns using maximum likelihood on the jump-diffusion return density. Estimate ρ from rolling realized spot–variance correlation. These parameters are relatively stable and do not require intraday recalibration.

Stage 2 — Implied calibration (online, every 5–15 min): Calibrate (κ, θ, v_0, σ_h) by minimizing the weighted sum of squared errors between Bates model prices and observed Deribit option mid-prices across strikes and maturities. Use the Levenberg-Marquardt or differential evolution optimizer. Penalize parameters outside the feasible region via soft constraints.

4.2 Calibration Objective

$$\min_{\Theta} \sum_i w_i \left(C_{market}(K_i,T_i) - C_{Bates}(K_i,T_i;\Theta)\right)^2 + \alpha \|\Theta - \Theta_{prior}\|^2$$

where weights w_i emphasize at-the-money and near-term options (highest information content), and the regularization term anchors parameters to the previous calibration to prevent overfitting and ensure temporal smoothness.

4.3 Calibration Validation

- Track calibration RMSE over time; flag regime changes when RMSE spikes above 2σ of its rolling mean.
- Monitor Feller condition: $2\kappa\theta > \sigma_h^2$ (ensures variance stays positive). Log warnings if violated.
- Cross-validate by holding out 20% of option strikes and checking out-of-sample pricing error.
- Compare implied vs realized jump frequency over rolling 30-day windows.

5. Pricing Engine

Methods for computing fair binary contract prices from calibrated parameters.

5.1 Primary Method: Fourier Inversion (COS Method)

The COS method (Fang & Oosterlee, 2008) is the recommended approach for production pricing due to its speed (O(N) complexity) and exponential convergence. Steps:

11. Compute the Bates characteristic function $\varphi(u)$ at the current calibrated parameters.
12. Truncate the integration domain to [a, b] using cumulants of the log-price distribution.
13. Expand the density in a cosine series and compute the binary option price as a weighted sum of cosine coefficients.
14. N = 64–128 terms typically suffices for 6+ digits of accuracy.

5.2 Validation Method: Monte Carlo with Variance Reduction

Used to validate COS prices and for complex path-dependent contract structures if Gemini introduces them. Implementation details:

- Euler-Maruyama or Milstein discretization of the joint (S, v) process with Poisson jump sampling.
- Full truncation scheme for the variance process (set v = max(v, 0) in the diffusion coefficient).
- Antithetic variates and control variates (use vanilla European option with known Bates price as control).
- 10^5–10^6 paths for benchmark accuracy; 10^4 paths for quick sanity checks.

5.3 Greeks and Sensitivities

Compute key risk measures for position management:

Greek	Method	Use
Delta (∂C/∂S)	Finite difference on COS pricer	Hedge ratio, spot exposure
Vega (∂C/∂v)	Finite difference on v_0	Volatility exposure
Theta (∂C/∂T)	Reprice at T-ε	Time decay monitoring
Jump sensitivity (∂C/∂λ)	Finite difference on λ	Sensitivity to jump regime changes

6. Trading Strategy

Signal generation, execution logic, and position management.

6.1 Mispricing Signal

For each active Gemini binary contract with strike K and expiry T, compute:

$$mispricing(K,T) = P_market(K,T) - P_fair(K,T;\Theta)$$

where P_market is the Gemini contract mid-price and P_fair is the Bates model fair value. Normalize by model-implied standard error:

$$z(K,T) = mispricing(K,T) / \sigma_model(K,T)$$

where σ_model is estimated from the sensitivity of the contract price to calibration parameter uncertainty (propagated via the Jacobian of the pricing function).

6.2 Entry Rules

- Enter long (buy contract) when z < -z_threshold (contract is underpriced).

- Enter short (sell contract) when z > +z_threshold (contract is overpriced).
- Default z_threshold = 2.0; optimize via backtest. Higher threshold = fewer trades, higher expected edge per trade.
- Require minimum model confidence: calibration RMSE below threshold and Feller condition satisfied.
- Require minimum time-to-expiry (e.g., > 2 hours) to avoid gamma risk near expiration.

6.3 Exit Rules

- Target exit when |z| < 0.5 (mispricing has mean-reverted).
- Stop-loss: exit if unrealized loss exceeds 2x the expected edge at entry.
- Time stop: flatten position if T_remaining < 30 minutes (binary gamma becomes unmanageable).
- Model stop: flatten all positions if calibration RMSE exceeds emergency threshold.

6.4 Position Sizing

Use a Kelly-criterion-derived fractional sizing approach:

$$f^* = (\hat{p} \times b - (1 - \hat{p})) / b$$

where $\hat{p}$ is the model-implied probability of profit and b is the contract payoff ratio. Apply a fractional Kelly multiplier (0.25–0.5) to account for model uncertainty. Maximum position size per contract: 5% of portfolio NAV. Maximum aggregate exposure: 25% of NAV.

7. Risk Management

Comprehensive risk controls and monitoring framework.

7.1 Model Risk

- Run parallel pricing under alternative models (Black-Scholes, Heston without jumps, SABR) and only trade when all models agree on the direction of mispricing.
- Monitor jump parameter stability: if λ or σ_j shift by > 50% in a single recalibration, halt trading and investigate.
- Maintain a model confidence score based on out-of-sample pricing error, Feller condition margin, and parameter stability.

7.2 Execution Risk

- Account for bid-ask spread on Gemini contracts; only trade when mispricing exceeds spread + transaction costs.

- Implement latency monitoring: if data feed lag exceeds 5 seconds, suspend signal generation.
- Rate-limit order submissions per Gemini API limits.

7.3 Portfolio Risk Controls

Control	Threshold	Action
Daily drawdown limit	-3% NAV	Halt trading for remainder of day
Weekly drawdown limit	-7% NAV	Halt trading; trigger manual review
Single contract max position	5% NAV	Reject new orders exceeding limit
Aggregate exposure	25% NAV	Reject new orders; reduce positions
Correlation concentration	>0.8 pairwise	Treat correlated contracts as single position

8. Implementation Architecture

System design, technology choices, and module decomposition.

8.1 Technology Stack

Layer	Technology	Rationale
Pricing engine core	Python (NumPy/SciPy) or C++ with Python bindings	COS method speed; SciPy optimizers for calibration
Data ingestion	Python asyncio + websockets	Concurrent connections to Gemini, Deribit
Time-series DB	TimescaleDB (PostgreSQL extension)	SQL interface, compression, continuous aggregates
Orchestration	Python event loop / Celery	Scheduled recalibration, signal evaluation
Execution	Gemini REST API via requests/httpx	Order management, balance queries
Monitoring	Grafana + Prometheus	Real-time dashboards for P&L, model health, latency
Backtesting	Custom Python framework or vectorbt	Historical strategy simulation

8.2 Module Decomposition

Module	Responsibility	Key Interfaces
data_ingest	WebSocket/REST connections, normalization, storage	Publishes tick data to message bus
calibrator	Bates model calibration (Stages 1 & 2)	Consumes option data; publishes parameter set Θ
pricer	COS method / Monte Carlo binary option pricing	Accepts (K, T, Θ); returns price + Greeks
signal_gen	Computes z-scores, applies entry/exit filters	Consumes contract prices + fair values; emits signals
executor	Order management, position tracking, fills	Consumes signals; interacts with Gemini API
risk_mgr	Drawdown checks, exposure limits, kill switch	Monitors executor; can override/halt
backtest	Historical simulation, performance analytics	Replays data_ingest output through calibrator→pricer→signal_gen

9. Development Phases

Sequential milestones with deliverables and acceptance criteria.

Phase 1: Foundation (Weeks 1–3)

- Implement the Bates characteristic function and COS pricing method for vanilla Europeans and binaries.
- Validate against known analytical results and published benchmarks (Bates 1996, Fang & Oosterlee 2008).
- Build Monte Carlo pricer as independent validation.
- Deliverable: Pricing library with unit tests, <0.01% error vs benchmarks.

Phase 2: Calibration Pipeline (Weeks 4–6)

- Build Deribit option data ingestion and IV surface construction.
- Implement Stage 1 (historical) and Stage 2 (implied) calibration.
- Test calibration stability across different BTC market regimes (trending, ranging, crash).
- Deliverable: Calibrator producing stable Θ with RMSE tracking and regime detection.

Phase 3: Data Infrastructure (Weeks 5–7)

- Deploy TimescaleDB; build ingestion for Gemini spot, Gemini contracts, Deribit options.
- Implement feature store: rolling realized vol, jump detection stats, IV surface snapshots.
- Deliverable: Live data pipeline with >99.5% uptime over 1 week of testing.

Phase 4: Signal & Strategy (Weeks 7–9)

- Implement z-score computation and entry/exit logic.
- Build position sizer with Kelly criterion and risk constraints.
- Deliverable: Signal generator producing actionable signals on live data (paper mode).

Phase 5: Backtesting (Weeks 8–11)

- Build backtesting framework replaying historical data through the full pipeline.
- Run backtest across multiple regimes; compute Sharpe, max drawdown, win rate, profit factor.
- Optimize z_threshold, Kelly fraction, and risk limits.
- Deliverable: Backtest report with statistical significance tests (bootstrap, walk-forward).

Phase 6: Paper Trading (Weeks 11–14)

- Deploy full system in paper trading mode against live Gemini contract prices.
- Monitor model calibration stability, signal quality, simulated P&L.
- Deliverable: 3+ weeks of paper trading data validating strategy performance.

Phase 7: Live Deployment (Week 15+)

- Deploy with minimal capital; gradually increase position limits.
- Implement automated monitoring, alerting, and kill switch.
- Deliverable: Production system with Grafana dashboards and automated risk controls.

10. Evaluation Metrics

Quantitative criteria for assessing model and strategy performance.

10.1 Model Quality

Metric	Target	Measurement
Calibration RMSE (IV points)	< 2.0 vol points	Across all strikes/maturities per calibration

Out-of-sample pricing error	< 3.0 vol points	20% held-out strikes
Calibration time	< 5 seconds	Per recalibration cycle
Binary price vs MC validation	< 0.5% absolute	Spot-check 10 contracts per day
Feller condition satisfaction	> 95% of calibrations	$2\kappa\theta / \sigma_h^2 > 1$

10.2 Strategy Performance

Metric	Target (annualized)	Notes
Sharpe ratio	> 1.5	After transaction costs
Maximum drawdown	< 15%	Peak-to-trough
Win rate	> 55%	Profitable trades / total trades
Profit factor	> 1.5	Gross profit / gross loss
Average edge per trade	> 1.5x transaction cost	Ensures profitability net of friction

11. Key Risks & Mitigations

Risk	Severity	Mitigation
Model misspecification (BTC dynamics not well-captured by Bates)	High	Run alternative models in parallel; monitor realized vs implied jump stats
Calibration instability in extreme vol regimes	High	Regularization, parameter bounds, Feller penalty; halt trading when RMSE spikes
Gemini contract illiquidity / wide spreads	Medium	Require minimum spread threshold for entry; use limit orders
Latency / stale data leading to stale signals	Medium	Heartbeat monitoring; auto-suspend if data lag > 5s
Overfitting z_threshold and Kelly fraction to backtest	Medium	Walk-forward validation; out-of-sample holdout; conservative Kelly fraction
Regulatory changes to prediction market contracts	Low	Monitor Gemini announcements; design for contract-agnostic architecture
Counterparty risk (Gemini insolvency)	Low	Limit capital deployed; diversify across venues if possible

12. Extension Roadmap

Future enhancements beyond the initial deployment.

- Regime-switching Bates model: Incorporate Markov-switching for κ, θ to handle structural breaks in BTC volatility dynamics.
- Machine learning residual model: Train a gradient-boosted model on calibration residuals to capture systematic model bias as a function of market microstructure features.
- Multi-asset expansion: Extend to ETH/USD or other crypto prediction contracts if Gemini offers them.
- Implied risk-neutral density extraction: Visualize the full model-implied density vs market-implied density to identify distributional mispricings beyond point prices.
- Bayesian calibration: Replace point estimation with MCMC posterior over Θ to obtain principled uncertainty quantification on fair prices.
- Market making mode: Instead of directional bets on mispricing, quote two-sided markets on contracts, earning the bid-ask spread while hedging with BTC spot or perpetuals.

Appendix A: Key References

Bates, D. (1996). Jumps and Stochastic Volatility: Exchange Rate Processes Implicit in Deutsche Mark Options. Review of Financial Studies, 9(1), 69–107.

Heston, S. (1993). A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options. Review of Financial Studies, 6(2), 327–343.

Fang, F. & Oosterlee, C.W. (2008). A Novel Pricing Method for European Options Based on Fourier-Cosine Series Expansions. SIAM Journal on Scientific Computing, 31(2), 826–848.

Merton, R. (1976). Option Pricing When Underlying Stock Returns Are Discontinuous. Journal of Financial Economics, 3(1–2), 125–144.

Carr, P. & Madan, D. (1999). Option Valuation Using the Fast Fourier Transform. Journal of Computational Finance, 2(4), 61–73.

Gatheral, J. (2006). The Volatility Surface: A Practitioner's Guide. Wiley Finance.

Appendix B: LLM Implementation Notes

Guidance for LLMs tasked with implementing components of this system.

Pricing module: Implement the Bates characteristic function exactly as specified in Section 2.3. Use the Fang-Oosterlee COS method for the binary option price integral. The integration

truncation bounds should be computed from the first four cumulants of the Bates log-price distribution. Validate against Monte Carlo with 10^6 paths.

Calibration module: Use scipy.optimize.differential_evolution for the global search, then polish with scipy.optimize.minimize (L-BFGS-B). The objective function is defined in Section 4.2. Enforce parameter bounds: $\kappa \in [0.1, 10]$, $\theta \in [0.01, 3]$, $\sigma_h \in [0.1, 5]$, $\rho \in [-0.99, 0.99]$, $v_0 \in [0.01, 3]$, $\lambda \in [0, 50]$, $\mu_j \in [-0.2, 0.2]$, $\sigma_j \in [0.01, 0.5]$.

Signal generation: The z-score formula in Section 6.1 requires propagating parameter uncertainty through the pricing function. Compute σ_model via delta-method: $\sigma_model^2 = J^T \Sigma J$, where J is the Jacobian of the pricing function w.r.t. Θ (computed by finite differences) and Σ is the covariance matrix of the calibrated parameters (from the inverse Hessian at the optimum).

Backtesting: Do not use future information. Calibrate only on data available at each historical timestamp. Account for execution lag (assume 1–2 second fill delay). Include transaction costs as a fixed percentage of contract notional. Use walk-forward optimization: train on 60 days, test on 20 days, roll forward.